CS 1312
Design and Analysis of Algorithms

Greedy Algorithms

Types of Problems

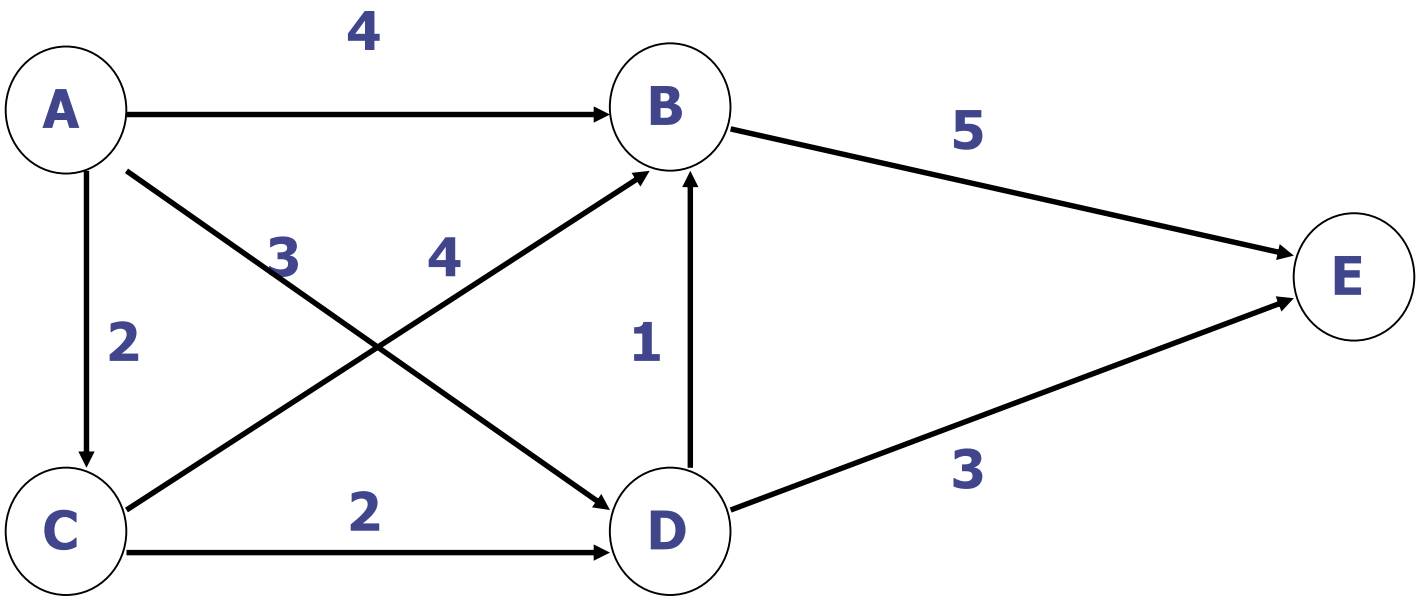
Problems we are trying to solve are basically of two kinds:

- **Decision problems:** trying to decide whether a statement is true or false.
  - They have either **yes** or **no** as the solution.

- **Optimization problems:** trying to find the solution with the **best** possible score according to some scoring scheme.
  - Maximization problems: trying to maximize a certain score.
  - Minimization problems: trying to minimize a cost function.

Types of Problems - example

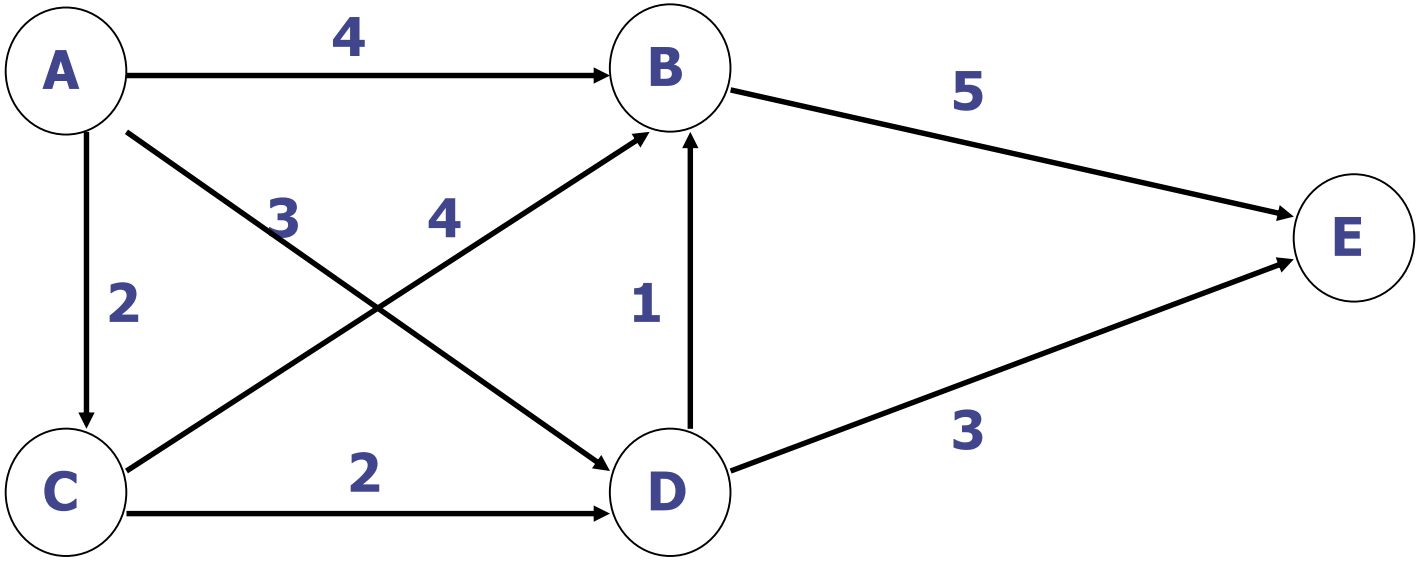
Find the shortest path from A to E



Decision Problem?
Optimization Problem?

Types of Problems - example

Does a path exist in G between A and E whose length is at most k?



Decision Problem?
Optimization Problem?

Optimization Problems

- Optimization: choosing the best element from a set of alternatives.
- An **optimization algorithm** is a numerical method or algorithm for finding a value x such that f(x) is as small (or as large) as possible, for a given function f, possibly with some constraints on x.
- For an optimization problem you want to find, not just *a* solution, but the best solution.

Algorithmic strategies

- Approaches to solving a problem
- Examples:
  - Greedy algorithms
  - Divide-and-conquer
  - Brute-force algorithms
  - Dynamic Programming
  - Backtracking

Greedy Algorithms

- Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step.
- A greedy algorithm is used to solve optimization problems by always making the choice that looks best at the moment.
  - It makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.
  - Hope local optimum lead to global optimum.

Greedy Algorithms

- Greedy algorithms do not always yield optimal solutions, but for many problems they do.
- The greedy method is quite powerful and works well for a wide range of problems.

Greedy Algorithms

- Examples of Greedy Algorithms :
  - Minimum spanning tree (Kruskal's algorithm)
  - Single-source shortest paths (Dijkstra’s algorithm)
  - Fractional knapsack problem
  - Task scheduling problem (Activity Selection)
  - Huffman Coding

Knapsack Problem

- The knapsack problem:
  - A thief robbing a store finds *n* items, *item*$_i$ is worth *i* dollars and w_i pounds, where *i* and w_i are integers.
  - The thief wants to take *as valuable a load as possible*, but he can carry at most *W* pounds in his knapsack, for some integer *W*.
  - Which items should he take?





Knapsack Problem

- Given some items, each item has some weight and some value.
- Goal: pack the knapsack to get the maximum total weight that we can carry is no more than some fixed number W.
- We must consider items weight and their values.

Item #	Weight	Value
1	1	8
2	3	6
3	5	5

Knapsack Problem

Two versions of the problem:

1. Fractional knapsack problem
   - Items are divisible: you can take any fraction of an item.
   - Some instances can be solved with greedy algorithm.



2. 0-1 knapsack problem
   - Items are indivisible: You either take an item or not.
   - Some instances can be solved with dynamic programming.



Fractional knapsack problem

- Redefine the problem:
  - Given a knapsack with maximum capacity W, and a set S consisting of n items.
  - Each item i has some weight w_i and benefit value b_i (all w_i and W are integer values)
  - Problem: How to pack the knapsack to achieve maximum total profit of packed items?

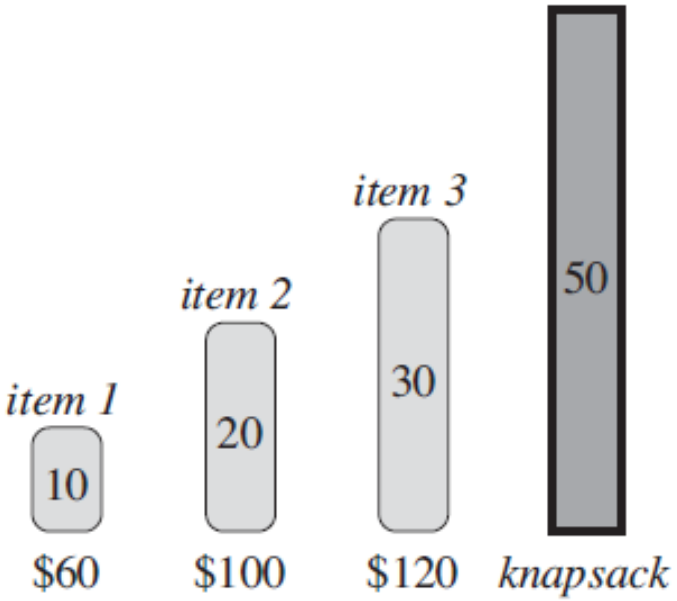
- It is called a "fractional" problem because a fraction of each item can be taken.

Fractional knapsack problem

Example:

Number of items n=3

Maximum capacity W=50



Fractional knapsack problem

Example:

Number of items n=3

Maximum capacity W=50

Item i	Weight w_i	benefit b_i
1	10	60
2	20	100
3	30	120

Fractional knapsack problem

Example:

Number of items n=3

Maximum capacity W=50

Item i	Weight w_i	benefit b_i	b_i/w_i
1	10	60	6
2	20	100	5
3	30	120	4

Fractional knapsack problem

Example:

Number of items n=3

Maximum capacity W=50

Item i	Weight w_i	benefit b_i	b_i/w_i
1	10	60	6
2	20	100	5
3	30	120	4

1-choose item 1

- Profit= 60
- Remaining weight = 50 – 10 =40

Fractional knapsack problem

Example:

Number of items n=3

Maximum capacity W=50

Item i	Weight w_i	benefit b_i	b_i/w_i
1	10	60	6
2	20	100	5
3	30	120	4

2-Choose item 2

- Profit= 60+100 = 160
- Remaining weight = 40 – 20 =20

Fractional knapsack problem

Example:

Number of items n=3

Maximum capacity W=50

Item i	Weight w_i	benefit b_i	b_i/w_i
1	10	60	6
2	20	100	5
3	30	120	4

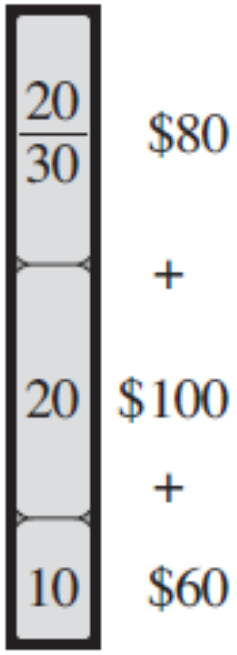
3-Choose 2/3 of item 3

- Profit= 160 + 2/3*120 = 240
- Remaining weight = 20 – 2/3*30 =0

Fractional knapsack problem

Example:

Solution:



Activity Selection and Scheduling

The problem to discuss is scheduling several competing activities that require **exclusive** use of a common resource, with a goal of selecting a **maximum-size** set of **mutually compatible** activities.

Assume $S=\{a_1, a_2, ..a_n\}$ activities each with a finite start time s_i and finish time f_i

Activities a_i and a_j are ***compatible*** if the interval $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap

In the activity-selection problem, we wish to select a maximum-size subset of mutually compatible activities. We assume that the activities are sorted in **monotonically increasing order of finish time**.

$$f_1 \leq f_2 \leq f_3 \leq \cdots \leq f_{n-1} \leq f_n$$

Example

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

A set $\{a_1, a_2, \ldots, a_{11}\}$ of activities. Activity a_i has start time s_i and finish time f_i.

To select a maximum-size subset of mutually compatible activities.

- The subset {a3, a9, a11} consists of mutually compatible activities but It is not a maximum subset.
- However {a1, a4, a8, a11} is a largest subset of mutually compatible activities
- Another largest subset is {a2, a4, a9, a11}.

Activity Selection and Scheduling Theorem

Consider any nonempty subproblem S_k, and let a_m be an activity in S_k with the earliest finish time. Then a_m is included in some maximum-size subset of mutually compatible activities of S_k.

Proof Let A_k be a maximum-size subset of mutually compatible activities in S_k, and let a_j be the activity in A_k with the earliest finish time. If $a_j = a_m$, we are done, since we have shown that a_m is in some maximum-size subset of mutually compatible activities of S_k. If $a_j \neq a_m$, let the set $A'_k = A_k - \{a_j\} \cup \{a_m\}$ be A_k but substituting a_m for a_j. The activities in A'_k are disjoint, which follows because the activities in A_k are disjoint, a_j is the first activity in A_k to finish, and $f_m \leq f_j$. Since $|A'_k| = |A_k|$, we conclude that A'_k is a maximum-size subset of mutually compatible activities of S_k, and it includes a_m. ■

In other words.. Since $f_m <= f_j$ then if a_j belongs to an optimal subset… then a_m belongs as well

Activity Selection and Scheduling

```
GREEDY-ACTIVITY-SELECTOR(s, f)
1  n = s.length
2  A = {a_1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {a_m}
7          k = m
8  return A
```

Time Complexity:

- Sorted list: O(n)
- Unsorted list: O(nlogn)

Activity Selection and Scheduling

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

A	m	k
$\{a_1\}$	2	1
	3	
$\{a_1, a_4\}$	4	4
	5	
	6	
	7	
$\{a_1, a_4, a_8\}$	8	8
	9	
	10	
$\{a_1, a_4, a_8, a_{11}\}$	11	11

GREEDY-ACTIVITY-SELECTOR (s, f)

```
1  n = s.length
2  A = {a_1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {a_m}
7          k = m
8  return A
```